Subcircuits and Debugging

I. Problem Description

As quantum hardware matures, a growing number of developers are writing quantum programs. Programming APIs such as Qiskit and Cirq allow developers to run these quantum programs on simulators or physical hardware. However, quantum computing is still early in its development and adoption, and quantum programming is an emerging skill. There are a wide variety of bugs that can occur in writing a quantum program, and most of them are specific to quantum computing rather than classical programming errors [1]. Some include incorrect qubit ordering, misplaced entangling gates, and swapping gate qubits (such as control and target for CNOT). These bugs can be difficult to detect because they often produce plausible but incorrect outputs without causing obvious errors. To make matters worse, quantum outputs are probabilistic, which means that even when the output is noticeably wrong, it is difficult to pinpoint exactly where the bug is. In classical software development, programmers will often employ breakpoints as one of the first steps in the debugging process. Breakpoints are extremely valuable because they allow programmers to view the state of the computation in the middle of the process. Huang and Martonosi note that "measurement disturbs the values of variables in a quantum computer," so breakpoints cannot be used in the same manner as in classical computing [2]. As a result, quantum developers lack a way to inspect a program's intermediate state.

Although classical-style breakpoints aren't possible, researchers have examined alternative approaches to inspect intermediate quantum states. One approach taken by Huang and Martonosi runs many shots of a quantum program up to a specified point to statistically test classical, superposition, or entanglement assertions [3]. Although less powerful than the classical equivalent, quantum assertions can be helpful in determining if a quantum state is highly likely to be incorrect. Only one assertion can be used at a time because the measurement destroys the quantum state necessary for the computation. Alternatively, Li et al. proposed projection-based assertions, which check whether the quantum state is within an expected subspace without disturbing the correct state, allowing execution to continue [4]. Two issues with these projection-based assertions are that most projections must be converted into additional gates to run on a quantum computer, and that for highly entangled states, the approach trades some checking accuracy for efficiency. Both approaches can indicate that a state is incorrect, but neither reveals the actual quantum state or allows execution to resume from an arbitrarily chosen point.

Using simulation allows developers to inspect intermediate simulated qubits without disturbing the states. This allows for easier discovery of circuit errors. However, capturing and storing the state introduces computational overhead. One example is system memory usage. An n-qubit state vector contains 2^n complex values, each requiring 16 bytes at double precision [5]. This becomes an issue for storing larger qubit state vectors in system memory. For example, a

30-qubit state vector would require approximately 16 GiB. The exponential growth of memory usage could become costly. Resuming execution from a breakpoint can also introduce major state preparation overhead. Preparing an n-qubit state requires exponentially many gates [6], potentially offsetting the benefit of avoiding earlier circuit stages.

The debugging process will use simulation-based breakpoint debugging in Qiskit with Qiskit Aer. The N-bit adder circuit will be divided into smaller subcircuits, with breakpoints placed after selected bit stages to inspect intermediate sum and carry values. At each breakpoint, the simulator will save the complete state vector without measuring the qubits, allowing the intermediate quantum state to be examined without measurement-induced collapse. If an error is detected, the saved state can be used to initialize a new circuit and execute the remaining subcircuits without rerunning the earlier stages. The resumed execution will be compared with uninterrupted execution to verify correctness. The project will also evaluate the overhead of state preparation, execution time, and memory usage as the adder size increases. This approach enables controlled debugging in simulation, although preparing a saved state may be computationally expensive and does not provide a direct way to recover an arbitrary quantum state from physical hardware.

Our correctness goal is to determine whether execution resumed from a breakpoint preserves the behavior of the original circuit. We will compare the prepared states to the captured states, compare execution with and without breakpoints, and verify circuit functionality against classical addition. We will also investigate the change of gate count and circuit depths when using exact state preparation and approximated state preparation while maintaining correctness. If approximated state preparation is made, we will measure the probability of obtaining correct results using different implementation methods.

II. Outline:

1) Determine solution
    a) Reversible gates: Use gates such as CNOT and Toffoli to build an N-bit adder while preserving the information needed for further computation.
    b) Quantum Fourier Transform (QFT): Explore a QFT-based adder and compare it with a ripple-carry adder to see which is more suitable for breakpoint debugging.
    c) Simulation-based debugging: Use Qiskit and Qiskit Aer to save and inspect the quantum state at selected points without measuring the qubits.
2) Determine a good circuit for evaluating the Quantum Computing breakpoint
    a) N-bit adder- Use an N-bit adder because its multiple stages and carry bits make it useful for testing intermediate results.

-Divide the adder into smaller stages and place breakpoints after selected stages to inspect intermediate sums and carries
- Evaluate different input combinations and adder sizes to determine whether the debugging method identifies errors correctly.

3) State preparation for resuming evaluating values at the breakpoints.
   State capture: Save the complete simulator statevector at each breakpoint so that the intermediate quantum state can be inspected and reused.
   a) Qiskit vs. BQSKit for state preparation
      - Investigate both tools for preparing the saved state in a new circuit, comparing state-preparation methods, circuit size, and execution overhead.
      - Execute the remaining subcircuits from the prepared state and compare the result against uninterrupted execution to validate correctness.
4) Determine methodology for measuring satisfactory values at breakpoints
   a) Verify intermediate values: Compare the sum and carry values at each breakpoint with the expected results.
   b) Check the quantum state: Examine state vectors and probability distributions to understand what is happening inside the circuit, including when the state is in superposition.
   c) Measure debugging accuracy: Check whether the method can identify errors and whether the resumed circuit produces the correct final result.
5) Reusing/Restarting the circuit
   a) Reuse saved states: Start execution from a saved breakpoint instead of running the entire circuit again.
   b) Measure the overhead: Record the time, memory, gate count, and circuit depth required for state preparation and resumed execution.
   c) Compare the trade-offs: Evaluate exact and optimized state-preparation methods to determine whether restarting from a breakpoint is worthwhile as the adder becomes larger.

III. Timeline:

10/22: Select project topic, generate solution ideas, generate repository, collect references
10/29: First group assignment homework due
11/17: Final group assignment homework due

IV. References

https://pypi.org/project/circuit-knitting-toolbox/
https://qiskit.github.io/qiskit-addon-cutting/
https://bqskit.lbl.gov/
https://hzhou.wordpress.ncsu.edu/files/2022/12/ASPLOS2020_camera_ready_copy.pdf
https://github.com/munich-quantum-toolkit/debugger

https://arxiv.org/pdf/2412.12269
https://bqskit.readthedocs.io/en/latest/
https://qiskit.github.io/qiskit-aer/index.html
https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.quantum_info.Statevector
https://ieeexplore.ieee.org/document/9825787 [1]
https://par.nsf.gov/servlets/purl/10085493 [2]
https://ieeexplore.ieee.org/document/8980343 [3]
https://dl.acm.org/doi/10.1145/3428218 [4]
https://qiskit.github.io/qiskit-aer/stubs/qiskit_aer.StatevectorSimulator.html (For system memory usage Nick [5])
https://arxiv.org/abs/1003.5760 (For setting up state CNOT cost Nick [6])
https://arxiv.org/abs/2205.01899 (Tool for debugging quantum circuits)
https://arxiv.org/abs/2412.12269 (Framework for debugging quantum programs)

V. GitHub Repository:
https://github.com/vardhanbhavsar10/CSC569_QC_Subcircuits_and_Debugging